Checkpoint dataflow, before and after

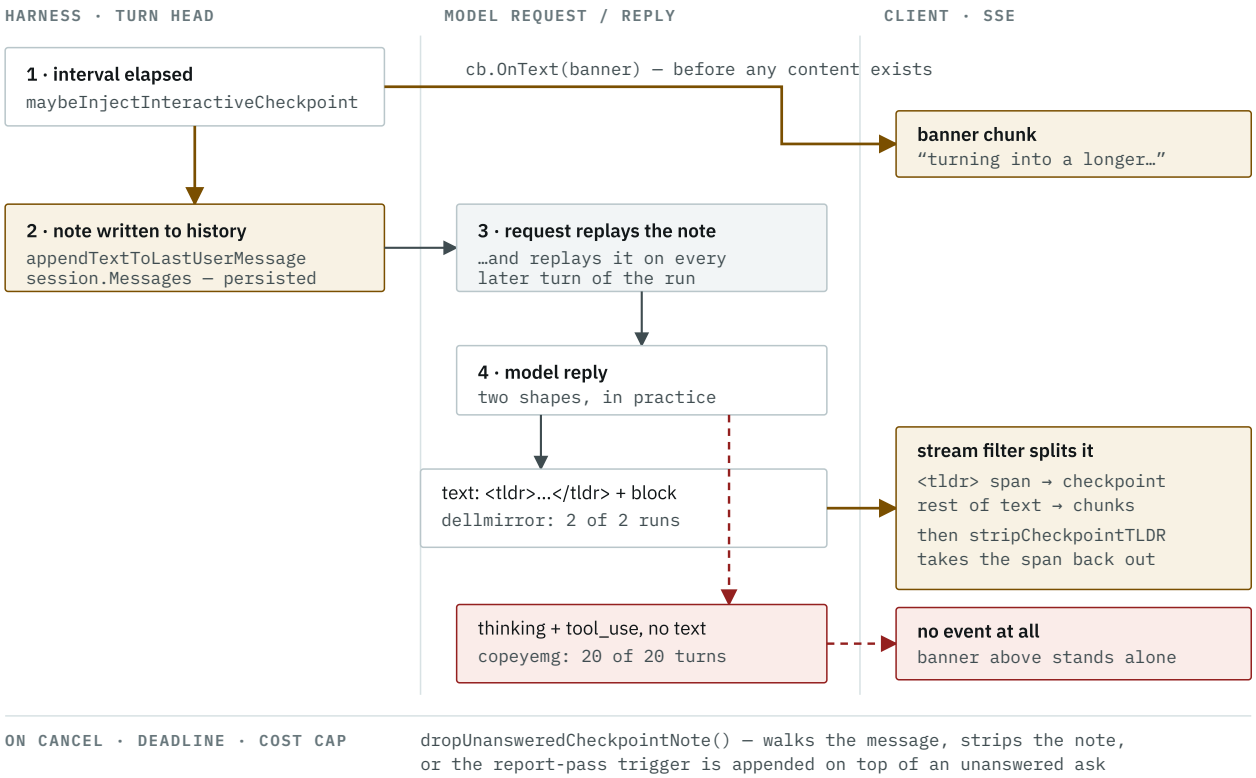
The long-run progress checkpoint used to be prose the harness parsed. It is now a tool call the harness renders. Same SSE contract on the wire — one emitter instead of three, and nothing written to the transcript to repair.

trigger: every 3 min of an interactive run net −292 lines wire contract: unchanged

01 · BEFORE

Prose, parsed out of the token stream

The harness wrote a note *into the session*, streamed the banner immediately, and then hoped the model would open its reply with a reserved `<tldr>` line and two headed sections. Three separate things emitted to the client, and a stream filter stood between the model and the transcript.

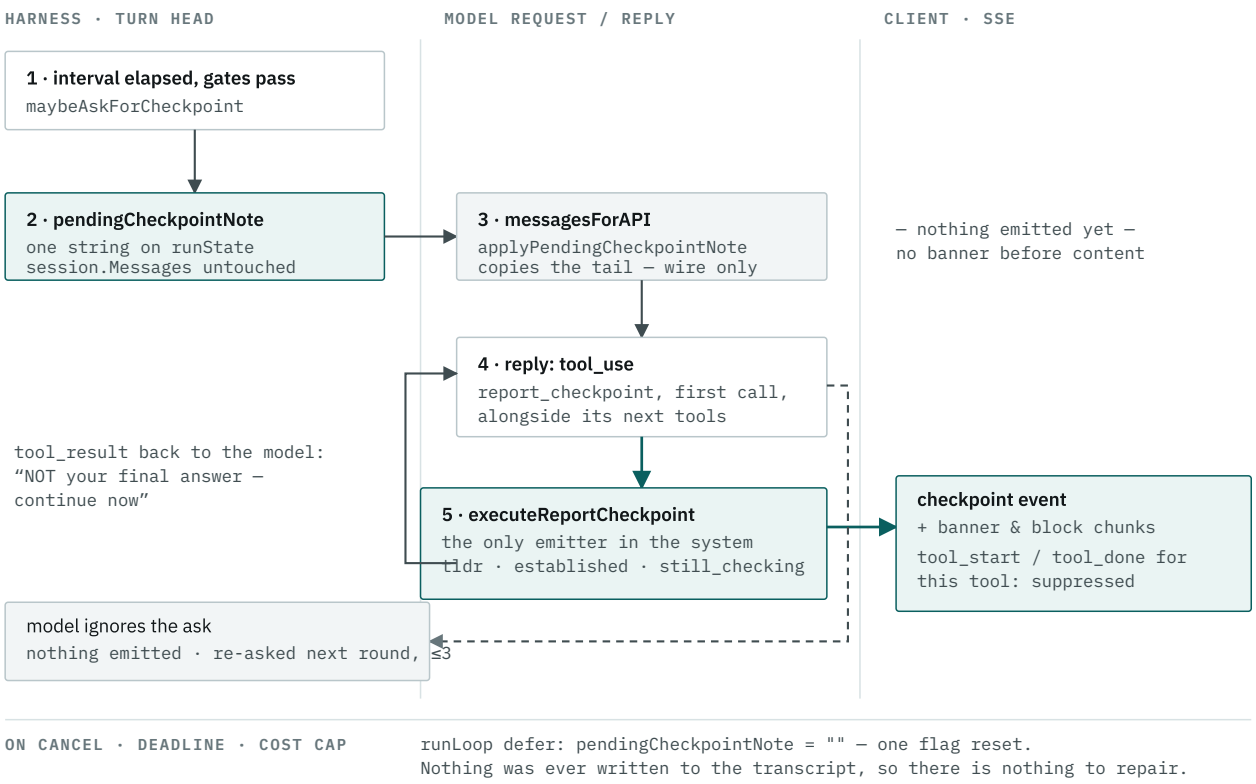


Three emitters: the banner (harness, before the reply), the status event (stream filter, only if the model wrote the reserved span), and the block (the model's own text). The note is persisted, so every bail-out path needs a repair pass — and when the model writes no text block, the banner has already gone out alone.

02 · AFTER

A tool call, rendered by the handler

The note goes on the outgoing request only. The model answers with a `report_checkpoint` call — which it does even on the turns where it writes no prose at all — and the handler is the single place anything reaches the client.

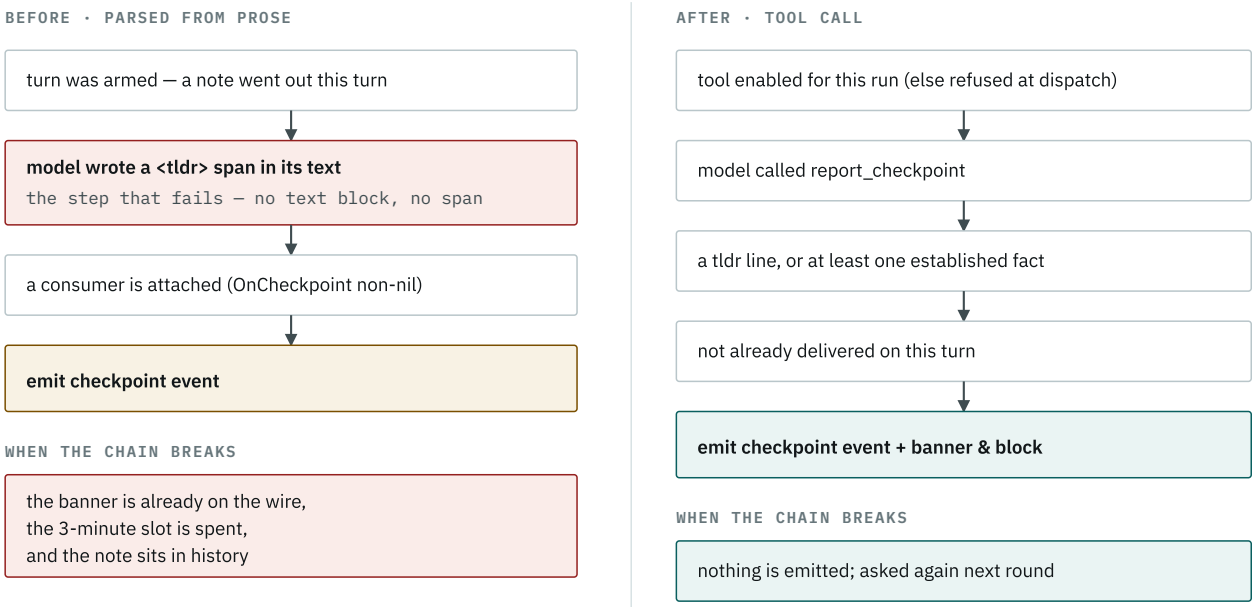


One emitter. The banner now arrives *with* the block, because both come from the handler — so “banner followed by silence” is not a state the system can be in. The note lives on one request and one `runState` field, which is what removes the repair pass rather than fixing it.

03 · THE EMISSION GATE

What decides that an SSE frame goes out

Both designs gate the `checkpoint` event behind a chain of conditions. The difference is where the chain can fail, and what the user sees when it does.



The gates read similarly; the asymmetry is in the failure. Before, the chain breaks *after* the banner has been sent and the slot consumed, so a failure is visible to the user as an unanswered promise. After, every emission is downstream of the tool call, so a failure is silent and recoverable — the ask simply repeats on the next tool round.

DECISION	BEFORE	AFTER
who emits	harness (banner) + stream filter (event) + model text (block) — three paths	`one` the tool handler, for all three
trigger lives	in `session.Messages`, replayed every later turn	on one request, and one `runState` field
event condition	a reserved span appeared in the token stream	a tool call arrived with usable fields
ordering	banner first, content later or never	banner and block together, after the reply completes
tool frames	n/a	suppressed for this tool only — no activity row beside the checkpoint
cancel · cap · deadline	repair pass rewrites the message to pull the note out	one `defer` clears a flag
model ignores it	banner stands alone; slot spent for 3 min	nothing shown; re-asked next round, ≤3 per window

What the UI receives is unchanged

Taken from the SSE wire test, which drives the real agent loop and the real writer. The `checkpoint` frame has the same shape it always had; what changed is that nothing else can produce one.

```
data: {"correlation_id":"toolu_a","name":"exec_sql","tool":"exec_sql","type":"tool_start"}
data: {"chunk":"\n\n---\n\n*◌ This is turning into a longer investigation…*\n\n","channel":"reasoning"}
data: {"chunk":"Established so far\n- pod-7 DB pool at 100/100 in use\n\nStill checking\n-
…","channel":"reasoning"}
data: {"type":"checkpoint","index":1,"turn":2,"elapsed_seconds":181,"tldr":"checkout 5xx is confined to
pod-7, whose DB pool is saturated"}
data: {"correlation_id":"toolu_b","name":"exec_sql","tool":"exec_sql","type":"tool_start"}
```

No `tool_start` or `tool_done` frame for `report_checkpoint` appears between them — the event *is* its output. The web bubble still finds the banner by its shape, the console writer still renders the one-line status, and the Slack status message still replaces the previous one.

05 · WHAT THE SHAPE CHANGE DELETES

Machinery that existed only to parse prose

Reserved markup

The `<tldr>` tags, the filter that carved them out of the token stream mid-delta, the strip that kept them out of history — and the leak when a later turn wrote the tag on a turn the filter wasn't armed for.

Reply-shape guards

The end-turn check for a reply that stopped *on* the progress block instead of continuing, and the nudge that pushed it back into the loop.

Transcript repair

`dropUnansweredCheckpointNote` and the message surgery it did at two bail-out sites, plus the predicate that had to recognise every note variant exactly.

What replaces them

One tool definition, one handler, one request-only string. A tool call can't end a turn and can't be half-parsed, so the failure modes those guards existed for don't arise.

06 · EVIDENCE

Why the shape change, and not a better note

Four note rewordings were measured against the exact request Opus 5.5 ignored on copeyemg, replayed through the dev gateway, one call per arm. The tool is the only arm that holds.

ARM	RESULT	READING
`prose note (shipped)`	0 / 1	thinking + 3 tool_use, no text — reproduces production
`note before tool_result`	400	a `tool_result` must immediately follow its `tool_use`
`shorter, imperative note`	0 / 1	thinking only; no text, no tools
`“text block first” rule`	1 / 3	complied once — luck, not a fix
`report_checkpoint tool`	`4 / 4`	always the first tool call, fields usable
`tool present, not asked for`	0 / 2	never called unprompted — so it can stay in the cached tool list

On a live local run at a 5-second interval the tool path delivered **3 of 3** checkpoints with zero tool frames and no harness note in the transcript; the prose path delivered **0 of 3** on the same workload.

One asymmetry worth keeping in view: prose compliance is deployment-shaped, not broken everywhere. On dellmirror it has now worked twice — UI-originated runs, small tool results, a model already narrating in text blocks. On the Slack-originated copeyemg run, with 4 KB tool results and twenty consecutive text-free turns, it never fired. The tool call removes the variance; the deletions are what make it worth doing now rather than later.